Practical 10

Aim :

Implement a web application using Microsoft Azure account as a cloud service by creating a web page and database. Also provide database connectivity with implemented webpage.

Objective :

To deploy a dynamic web application on Microsoft Azure App Service (PaaS), to provision an Azure SQL Database as the backend data store, and to establish live database connectivity from the hosted web page so that user data submitted through the browser is inserted into and retrieved from the cloud database.

Theory :

Microsoft Azure is a public cloud computing platform that provides Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). In this practical we use the PaaS model, where Azure manages the underlying operating system, runtime, and scaling, and we only deploy our application code and data.

- **Azure App Service** — A fully managed PaaS offering for hosting web applications. It supports multiple runtimes (PHP, Node.js, Python, .NET, Java), provides built-in auto-scaling, load balancing, HTTPS, and a public URL of the form https://<appname>.azurewebsites.net.
- **Azure SQL Database** — A managed relational database service (Database as a Service) based on Microsoft SQL Server. Azure handles backups, patching, and high availability automatically.
- **Database Connectivity** — The web page communicates with the database through a connection string containing the server name, database name, username, password, and encryption settings. The Azure SQL server firewall must be configured to allow connections from Azure services.
- **Architecture Flow** — User → Browser (web page form) → Azure App Service (PHP code) → Azure SQL Database (table) → results rendered back on the web page.

Requirements :

Component	Specification
Cloud account	Microsoft Azure account (free trial / student subscription)
App hosting	Azure App Service (Runtime: PHP 8.x, OS: Linux)
Database	Azure SQL Database (Basic tier) with SQL Server logical server
Local tools	Any text editor (VS Code), web browser, Azure CLI (optional)
Application files	index.php (web page + connectivity logic)

Step-by-Step Procedure :

Step 1: Sign in to the Azure Portal

- Open https://portal.azure.com and sign in with the Azure account.
- Confirm that an active subscription (e.g., Azure for Students / Free Trial) is available.

Step 2: Create a Resource Group

- Search for **Resource groups** → **Create**.
- Name: webapp-rg; Region: Central India → **Review + Create**.

Step 3: Create the Azure SQL Database

- Search for **SQL databases** → **Create**.
- Resource group: webapp-rg; Database name: studentdb.
- Under **Server**, click **Create new**:
  - Server name: studentserver149 (globally unique)
  - Authentication: SQL authentication
  - Admin login: azureadmin; Password: P@ssword123
- Compute + storage: **Basic** tier → **Review + Create** → **Create**.

Step 4: Configure the SQL Server Firewall

- Open the SQL server studentserver149 → **Networking**.
- Set **Allow Azure services and resources to access this server** to **Yes** (required so App Service can connect).
- Optionally add the client IP to run the Query editor from the portal → **Save**.

Step 5: Create the Database Table

- Open the database studentdb → **Query editor** → sign in with azureadmin.
- Execute the following SQL:

```
CREATE TABLE Students (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(50),
    Email NVARCHAR(50)
);
```

Step 6: Create the Azure App Service (Web App)

- Search for **App Services** → **Create** → **Web App**.
- Resource group: webapp-rg; Name: studentwebapp149.
- Publish: **Code**; Runtime stack: **PHP 8.2**; OS: **Linux**.
- Pricing plan: **Free F1** → **Review + Create** → **Create**.

- After deployment, the app is live at https://studentwebapp149.azurewebsites.net.

Step 7: Develop the Web Page with Database Connectivity

- Create index.php containing the HTML registration form and the PHP code that connects to Azure SQL using the PDO driver.

Step 8: Deploy the Web Page to Azure

- In the App Service, open **Development Tools → Advanced Tools (Kudu) → Go**, or use VS Code with the *Azure App Service* extension.
- Upload / deploy index.php to /site/wwwroot/.

Step 9: Test the Application

- Browse to https://studentwebapp149.azurewebsites.net.
- Fill the form (Name, Email) and click **Submit** — the record is inserted into Azure SQL Database.
- The page displays all records fetched from the Students table, confirming live connectivity.

Code (index.php) :

```
<?php
// ---------- Step A: Database connection details ----------
$server     = "studentserver149.database.windows.net,1433";
$database = "studentdb";
$username = "azureadmin";
$password = "P@ssword123";

try {
    // PDO connection string for Azure SQL (SQL Server driver)
    $conn = new PDO(

"sqlsrv:server=$server;Database=$database;Encrypt=true;TrustServerCertificate=false",
        $username,
        $password
    );
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $message = "Connected to Azure SQL Database successfully!";
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}

// ---------- Step B: Insert record when the form is submitted ----------
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST["name"];
    $email = $_POST["email"];

    $sql = "INSERT INTO Students (Name, Email) VALUES (:name, :email)";
    $stmt = $conn->prepare($sql);
    $stmt->bindParam(':name', $name);
    $stmt->bindParam(':email', $email);
    $stmt->execute();

    $message = "Record inserted successfully into Azure SQL Database!";
```

```
}

// ---------- Step C: Fetch all records from the database ----------
$stmt = $conn->query("SELECT ID, Name, Email FROM Students");
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
?>

<!DOCTYPE
html> <html>
<head>
    <title>Azure Student Registration</title>
</head>
<body>
    <h2>Student Registration - Hosted on Microsoft Azure</h2>
    <p><?php echo $message; ?></p>

    <!-- Web page form: data goes to Azure SQL Database -->
    <form method="POST" action="index.php">
        Name: <input type="text" name="name" required><br><br>
        Email: <input type="email" name="email"
        required><br><br> <input type="submit" value="Submit">

    </form>

    <h3>Records Stored in Azure SQL Database</h3>
    <table border="1" cellpadding="6">
        <tr><th>ID</th><th>Name</th><th>Email</th></tr>
        <?php foreach ($students as $row): ?>
        <tr>
            <td><?php echo $row["ID"]; ?></td>
            <td><?php echo $row["Name"]; ?></td>
            <td><?php echo $row["Email"]; ?></td>
        </tr>
        <?php endforeach; ?>
    </table>
</body
>
</html
>
```

Working of the Code :

- **Step A** builds a PDO connection to studentserver149.database.windows.net on port 1433 with encryption enabled — this is the database connectivity layer between the web page and Azure SQL.
- **Step B** executes when the user submits the form: a parameterized INSERT statement safely writes the Name and Email into the Students table (parameter binding prevents SQL injection).
- **Step C** runs a SELECT query and fetches all rows, which are then rendered as an HTML table on the web page — proving the round trip Browser → App Service → Azure SQL Database → Browser.

Output :

1. The web application loads at the Azure public URL and confirms connectivity:

```
Student Registration - Hosted on Microsoft Azure

Connected to Azure SQL Database successfully!

Name: [ Rahul Sharma    ]
Email: [ rahul@gmail.com  ]  [ Submit ]
```

2. After clicking Submit, the record is inserted and all rows stored in the cloud database are displayed:

```
Record inserted successfully into Azure SQL Database!

Records Stored in Azure SQL Database
ID  | Name      | Email ----|-------------|--
---------------  1      |  Rahul  Sharma  |
 rahul@gmail.com 2    | Priya  Patel   |
 priya@gmail.com
```

3. The same rows are visible in the Azure portal Query editor for studentdb , verifying that the data submitted from the web page is physically stored in the Azure SQL Database.

Conclusion :

In this practical, a complete web application was implemented on Microsoft Azure using the PaaS cloud service model. A web page (student registration form) was hosted on Azure App Service, an Azure SQL Database was provisioned as the cloud data store, and live database connectivity was established using a PDO connection string with the SQL Server driver. Data submitted through the web page was successfully inserted into and retrieved from the Azure cloud database, demonstrating end-to-end integration of a web front end with a managed cloud database — the standard architecture of real-world cloud-hosted web applications.